

오픈소스SW의 이해

CHAPTER 3

SEMESTER | 2026-2

박성미

목차

Table of Contents



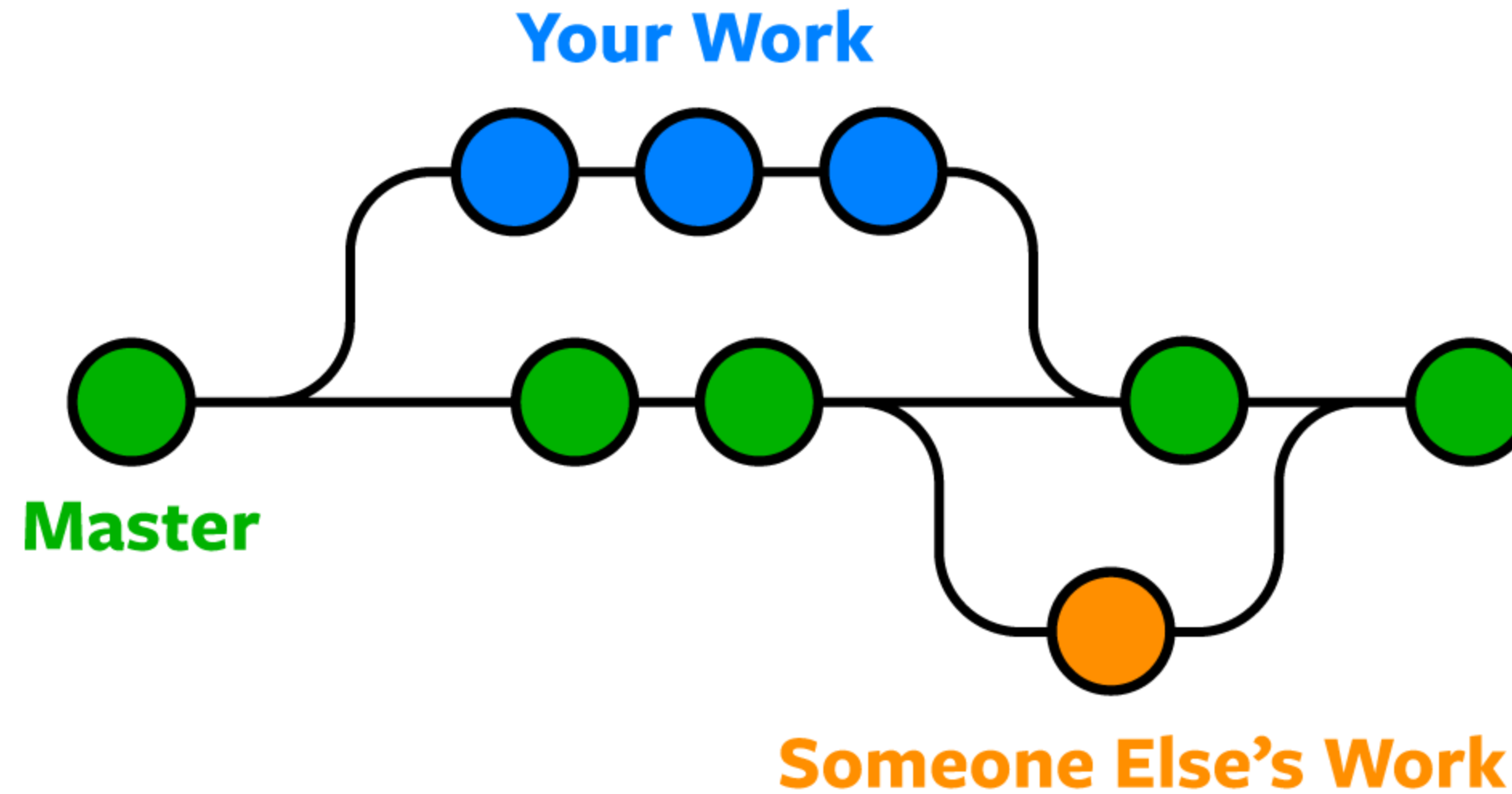
01. Git Branch 기초

02. Branch 변경하기

Git Branch 기초

한림대학교

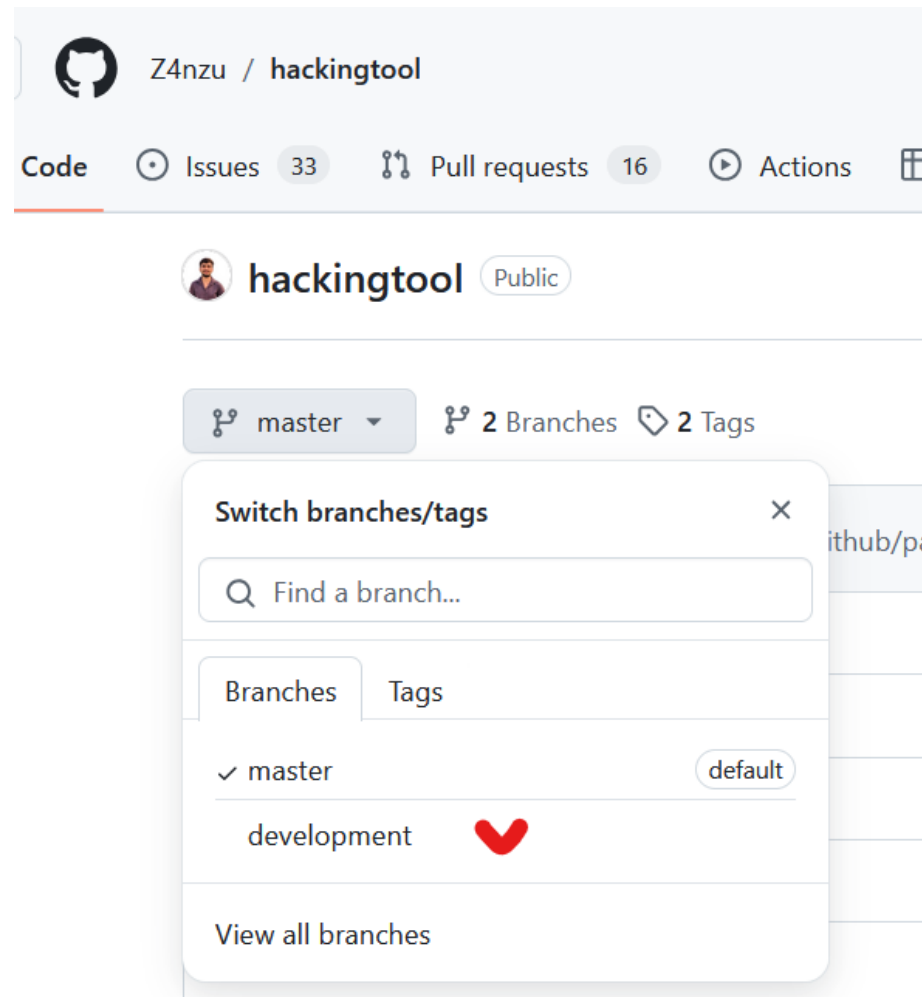
Git Branch 기초



❖ Git Branch란?

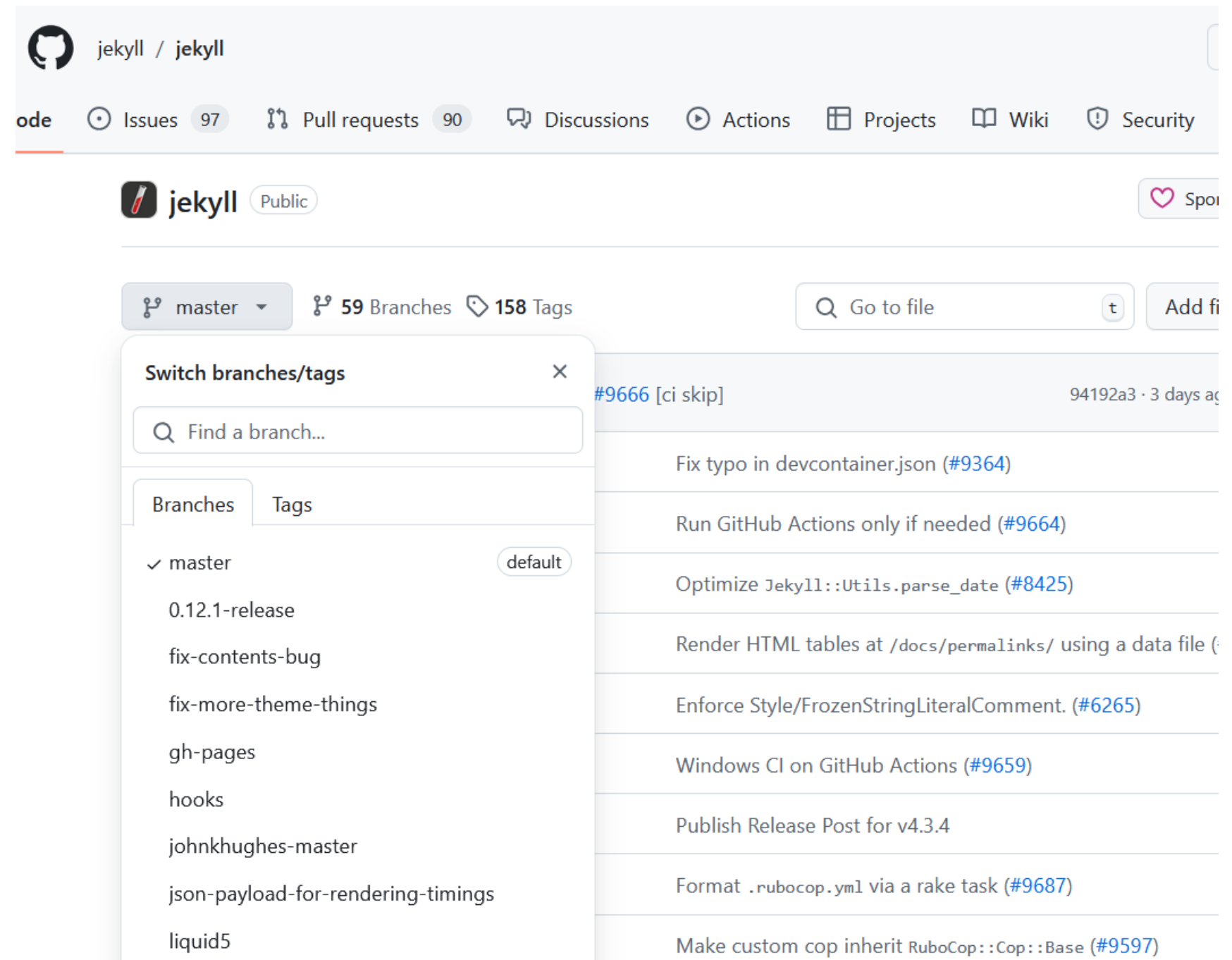
- 처음 작업한 저장소(main/master)를 여러 개 복사하여
각 작업자마다 다른 이름으로 버전관리하는 방법 (보통 feature 별로 관리)

Git Branch 기초



주요 브랜치 종류

- Master: 제품 출시용 브랜치
- Develop: 다음 버전 개발용 브랜치
- Feature: 기능 개발용 브랜치
- Release: 출시 준비용 브랜치
- Hotfix: 긴급 버그 수정용 브랜치



Git Branch 기초

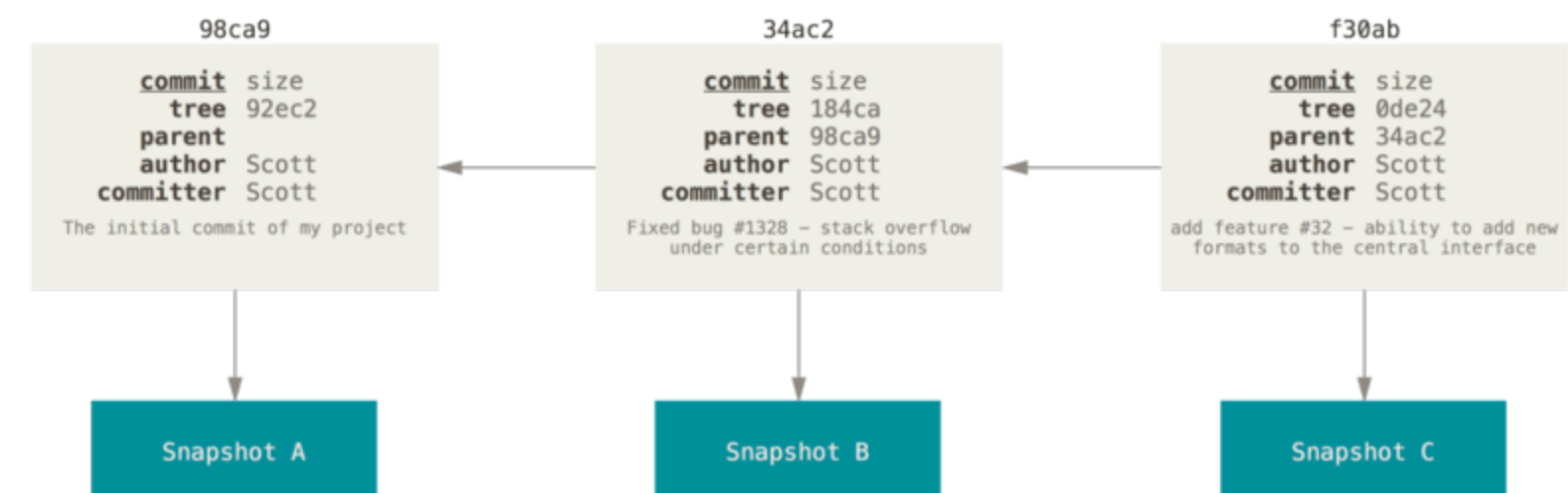
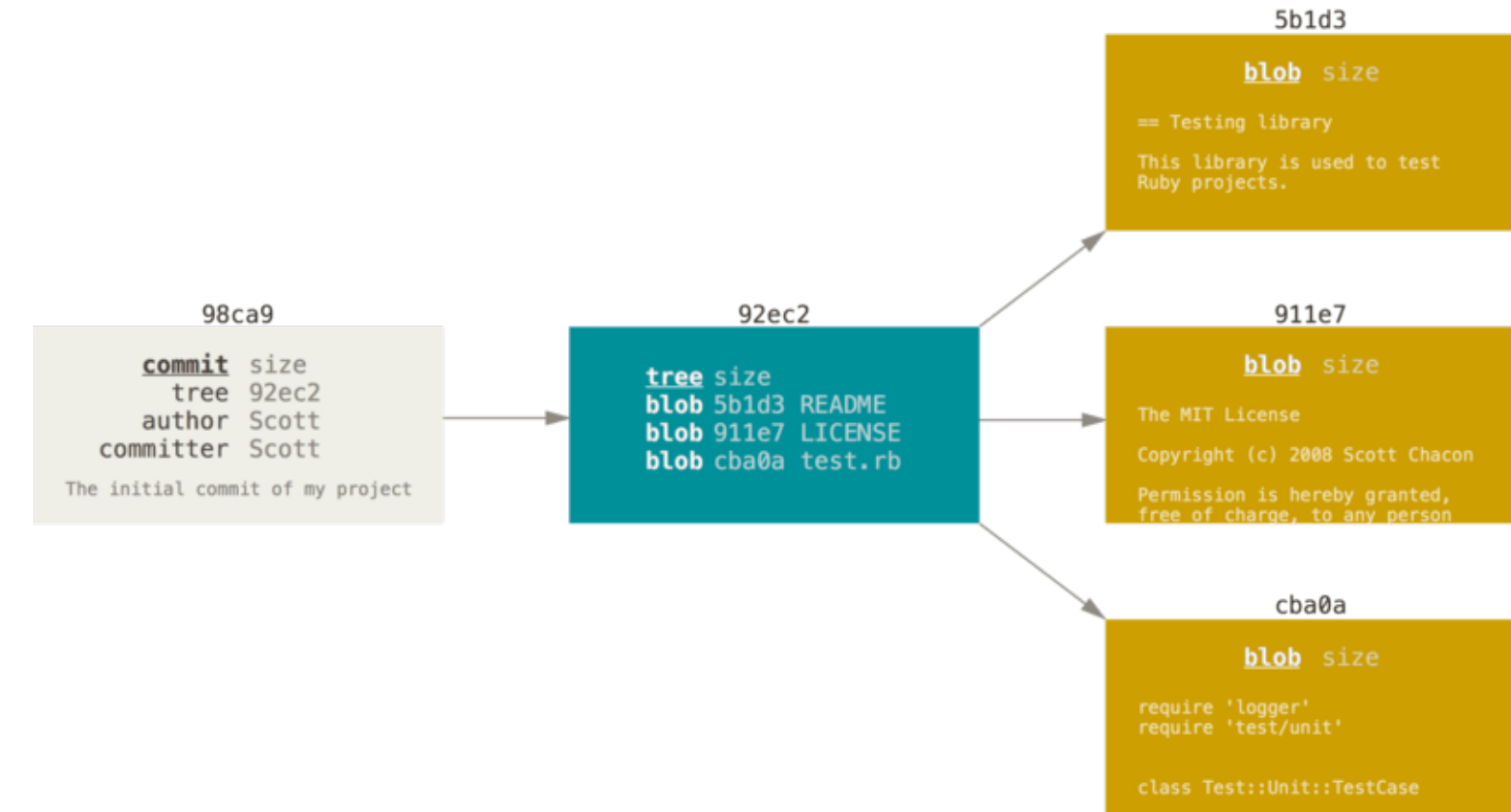
1 Git Branch 란?

❖ 브랜치:

- ✓ 커밋 사이를 이동할 수 있는 **포인터**
- ✓ 독립적인 작업 흐름을 만들어내는 도구
- ✓ 하나의 프로젝트에 여러 개발자가 동시에 작업 가능

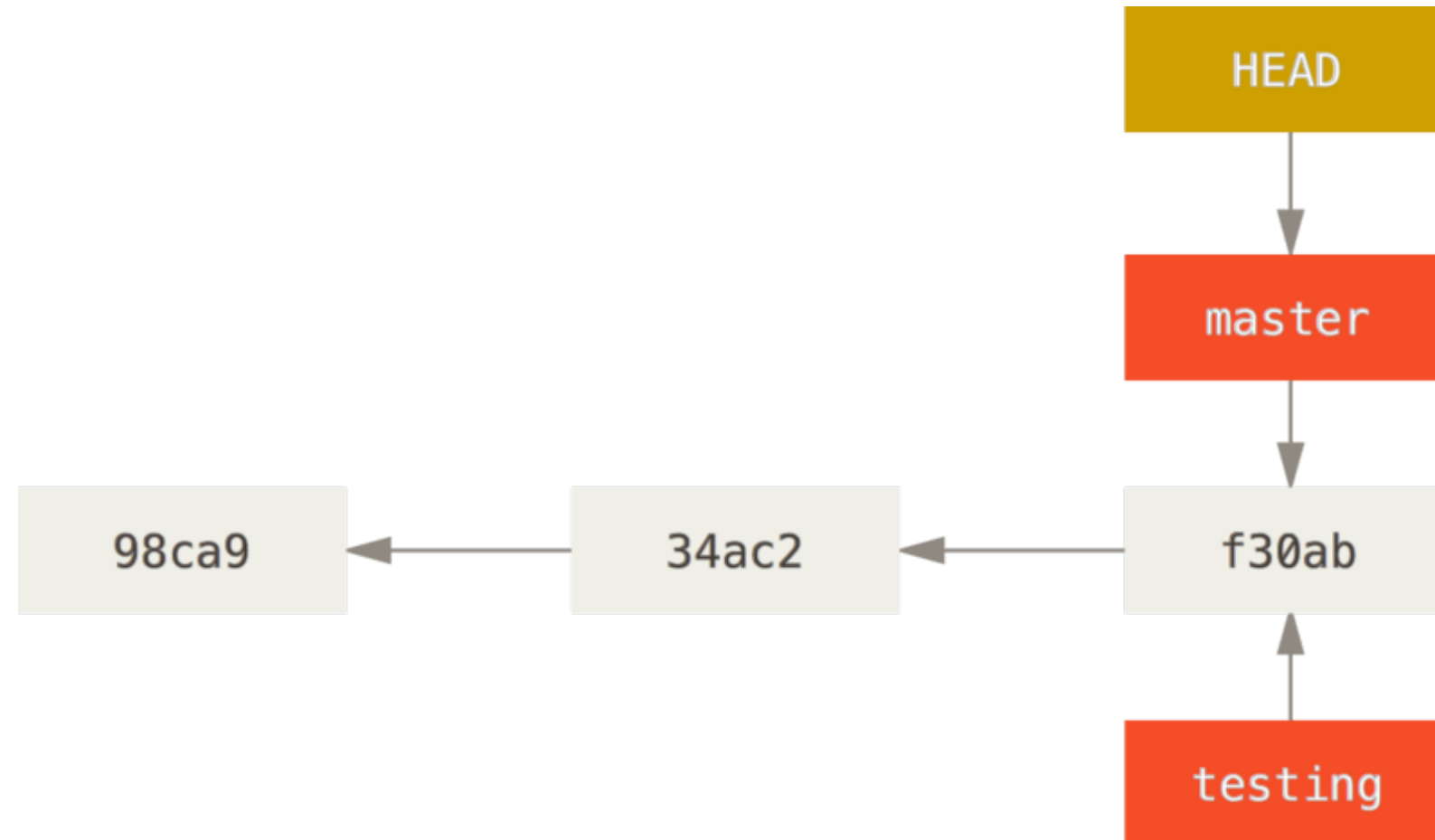
❖ 주요 특징:

- 메인 코드에 영향을 주지 않고 별도 공간에서 작업 가능
- 각 브랜치는 독립된 작업 공간을 가짐
- 작업 완료 후 메인 브랜치에 병합(merge) 가능

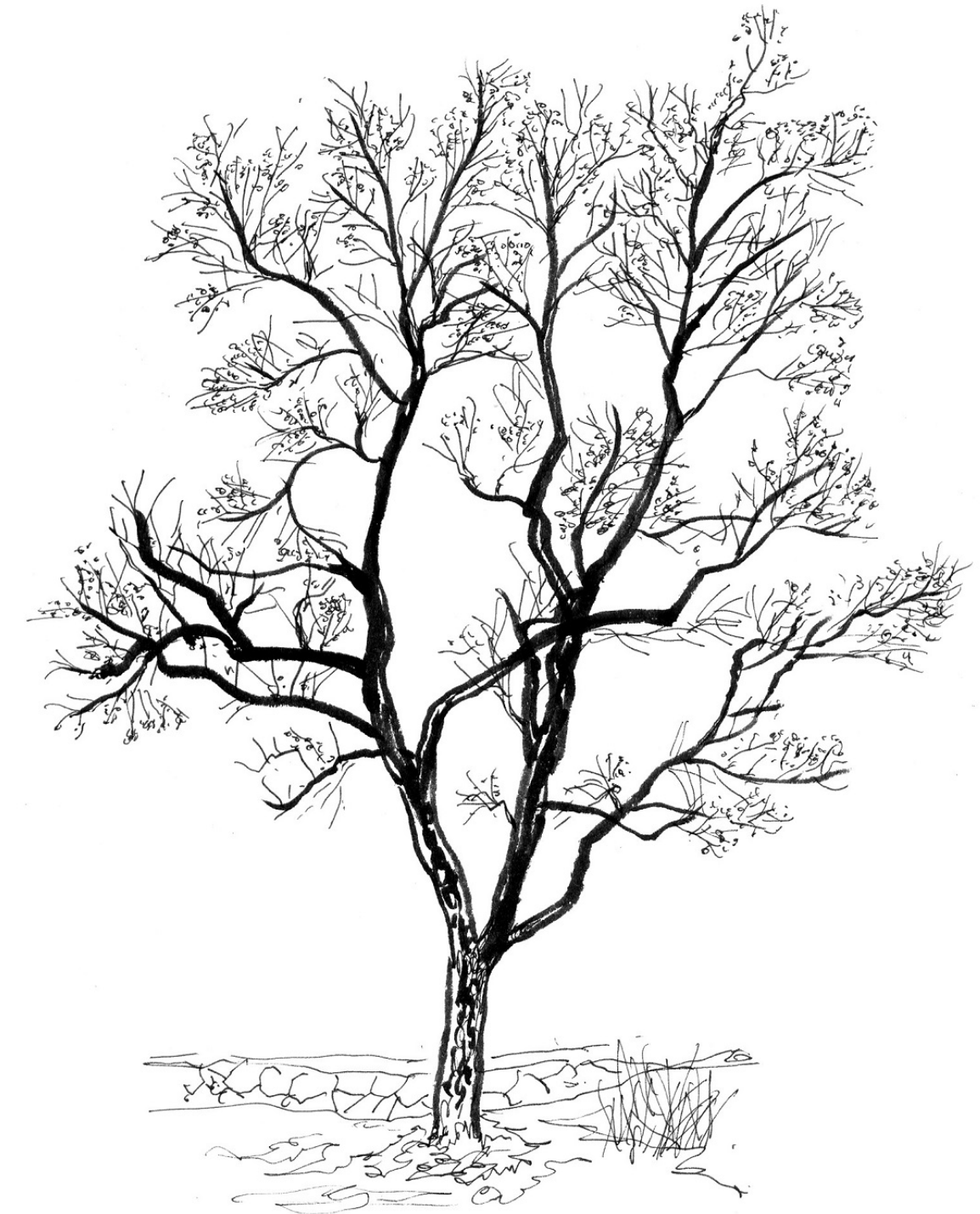


※ Git의 Snapshot 저장 방식

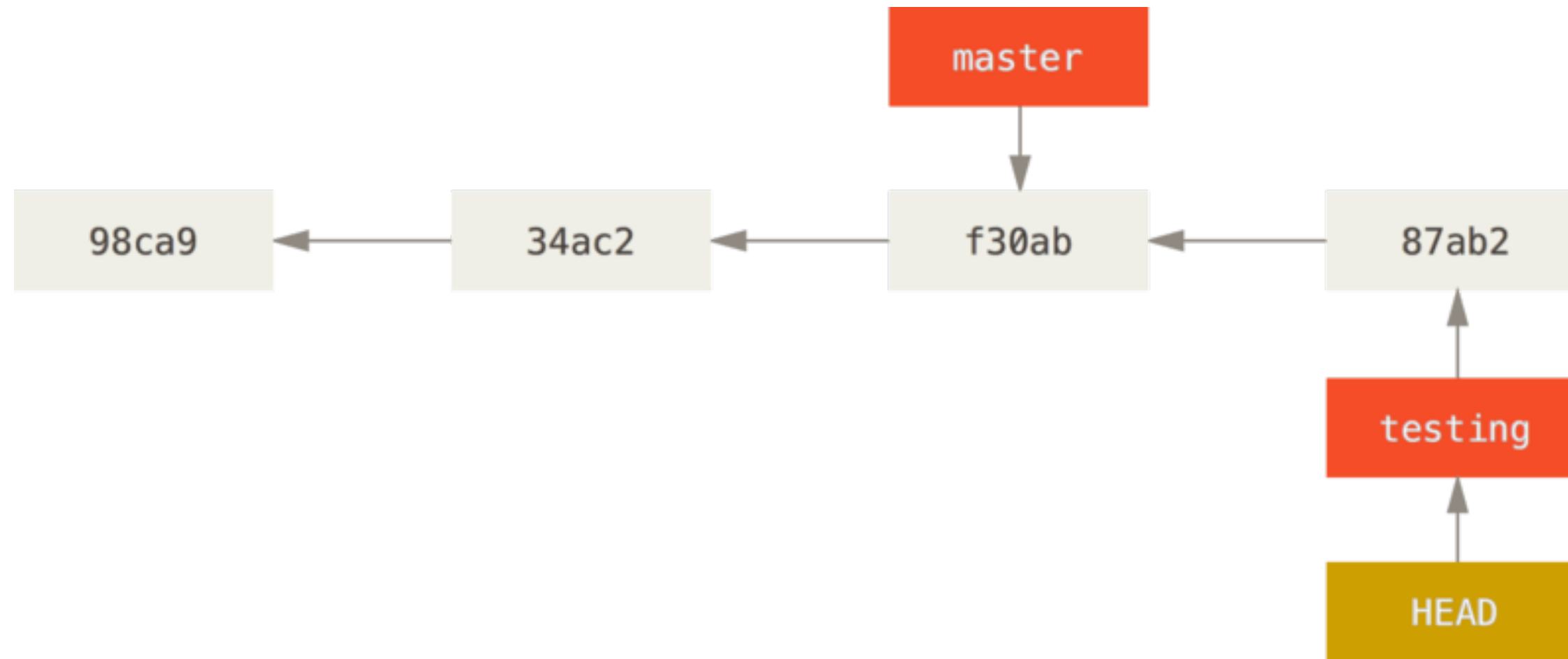
- 파일의 변경사항을 **스냅샷** 형태로 저장
스냅샷: 특정 시점의 파일과 폴더 상태를 그대로 캡처
- 커밋을 실행할 때마다 전체 프로젝트의 스냅샷이 저장
- 마지막 커밋의 스냅샷만 완전히 저장
- 이전 버전들은 변경된 부분(델타)만 저장



- 브랜치(가지) 생성하기:
 - 가지 1 (master) 에서 작업을 하다가
 - 가지 2 (testing) 로 되돌아가기
 - 각 작업 내용은 독립적으로 존재

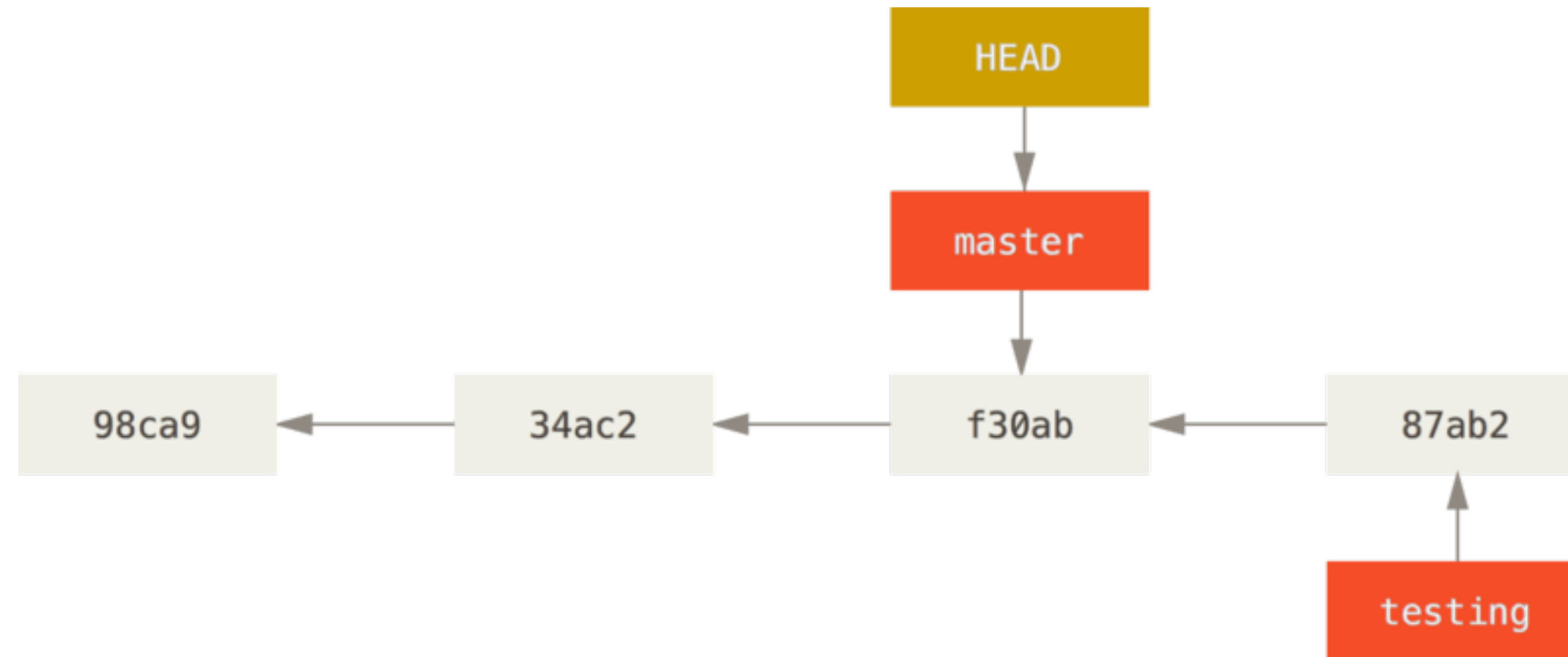


참느릅나무



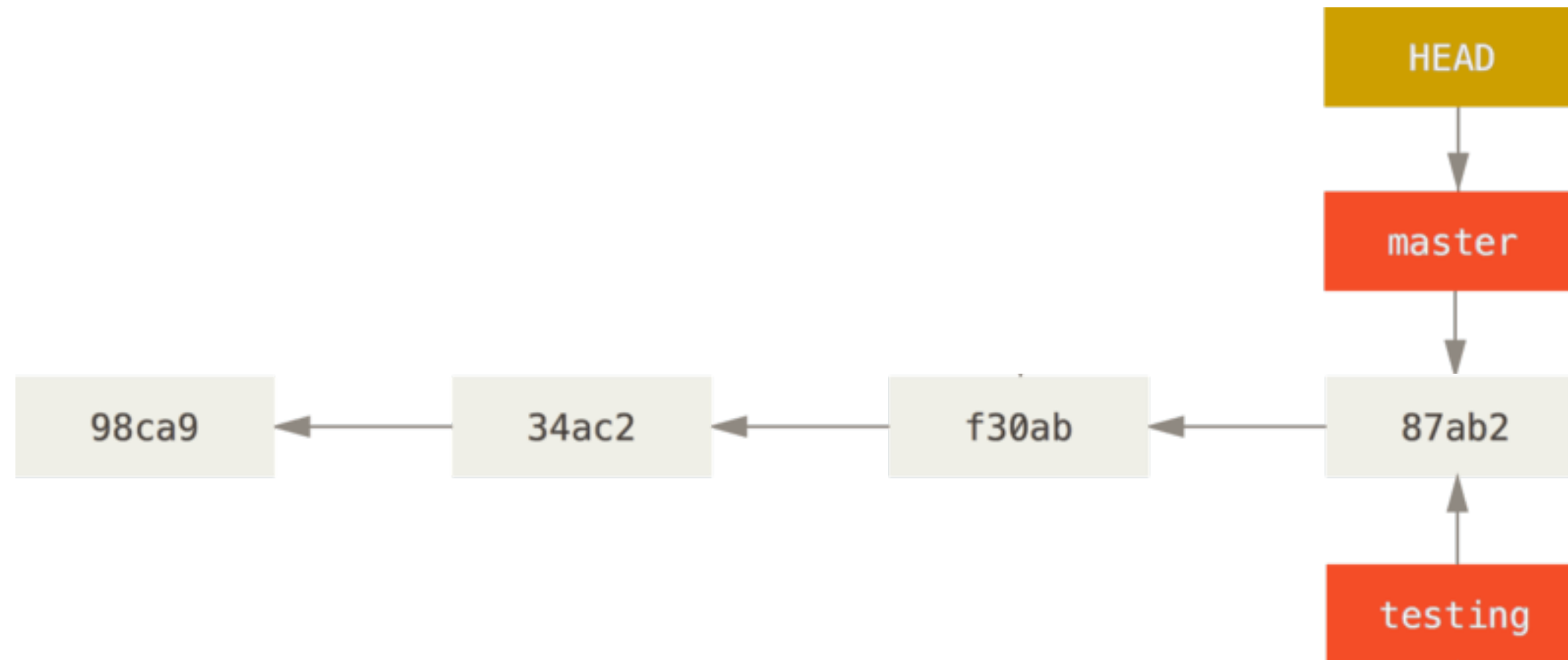
- 브랜치(가지) 생성하기:
 - 가지 (testing)에서 새로운 작업 내용 생성
 - Master (main) 브랜치는 변화없음

Git Branch 기초

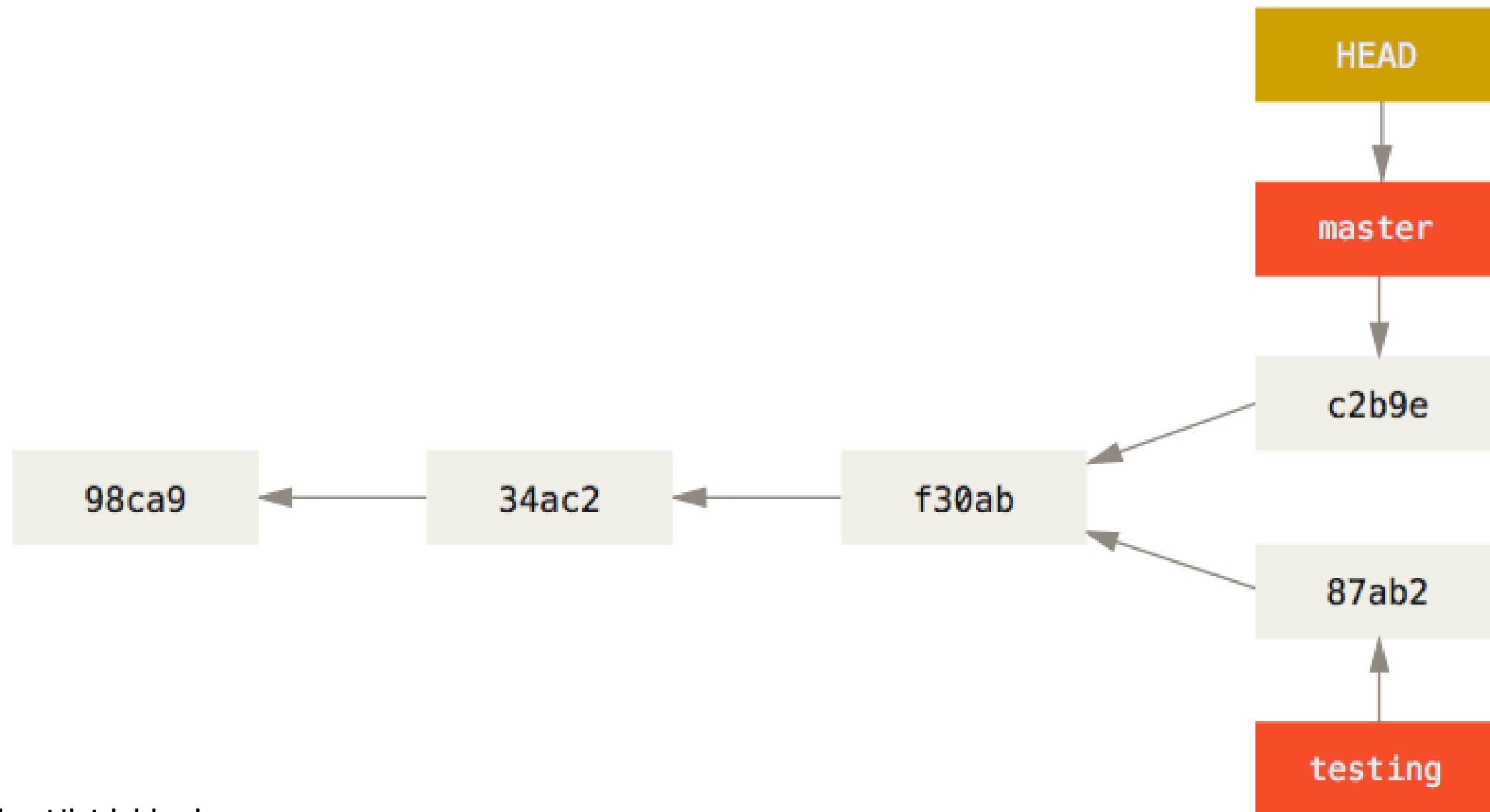


- 브랜치(가지) 생성하기:
 - Testing 브랜치 내용을 확인 후
 - Master 브랜치로 넘어가기
 - Master 브랜치를 최신 testing 내용으로 업데이트 (merge, 병합) 예정

Git Branch 기초

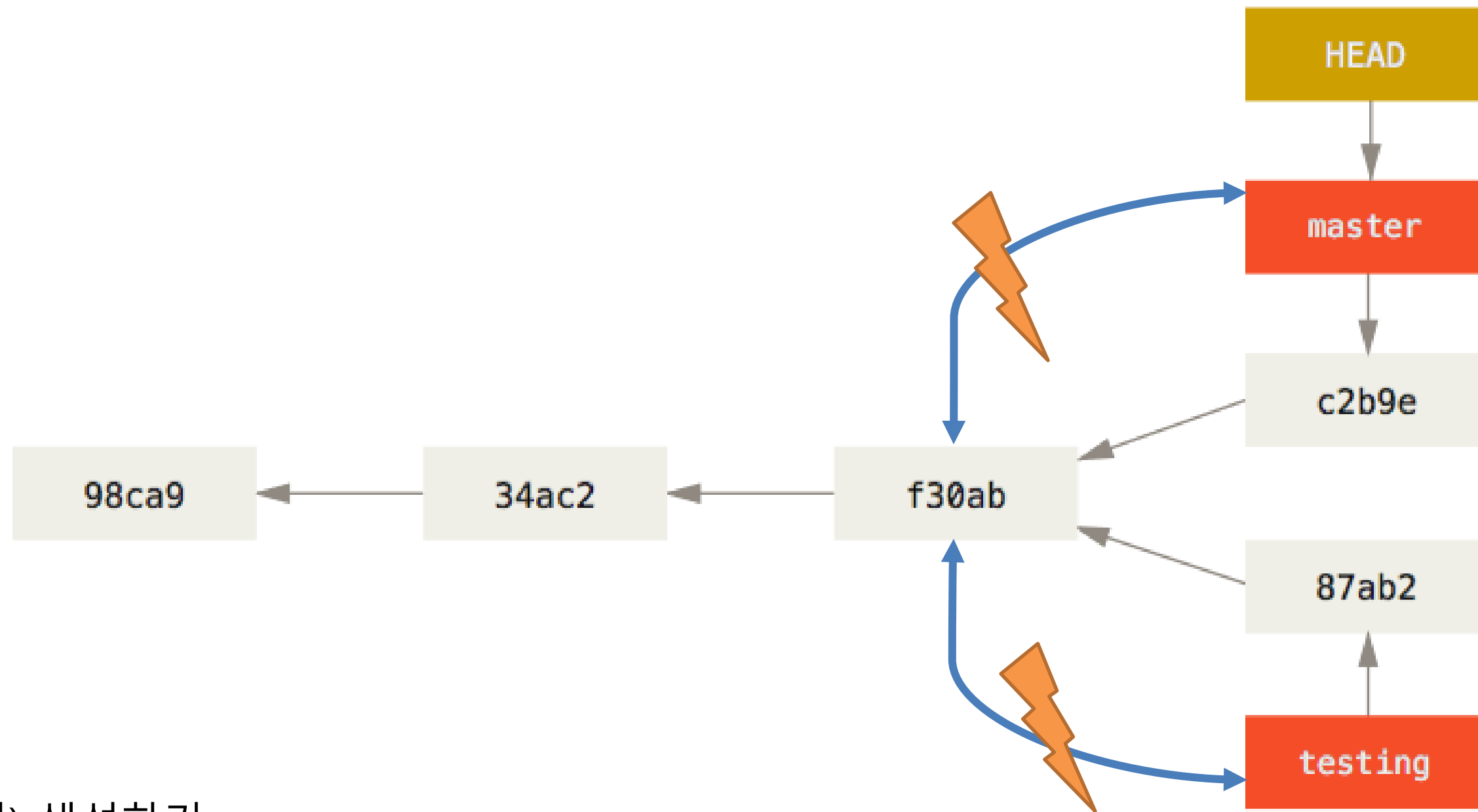


- 브랜치(가지) 생성하기:
 - 병합 (merge) 진행
 - Master에서 새로운 내용이 없기 때문에 간단 병합
 - ➔ **Fast-forward (빨리감기 병합):** 단순 포인터만 움직임



- 브랜치(가지) 생성하기:
 - f30ab에서 갈라진 브랜치
 - Master에서 새로운 내용 추가
 - Testing에서 새로운 내용 추가

Git Branch 기초



- 브랜치(가지) 생성하기:
 - 충돌 발생 가능!!!
 - ➔ 수정한 부분이 독립적일 경우: 자동으로 병합
 - ➔ 수정한 부분이 겹치는 경우: 개발자가 수동으로 선택을 해야 함 (충돌 해결)

Branch 변경하기

한림대학교

Branch 변경하기

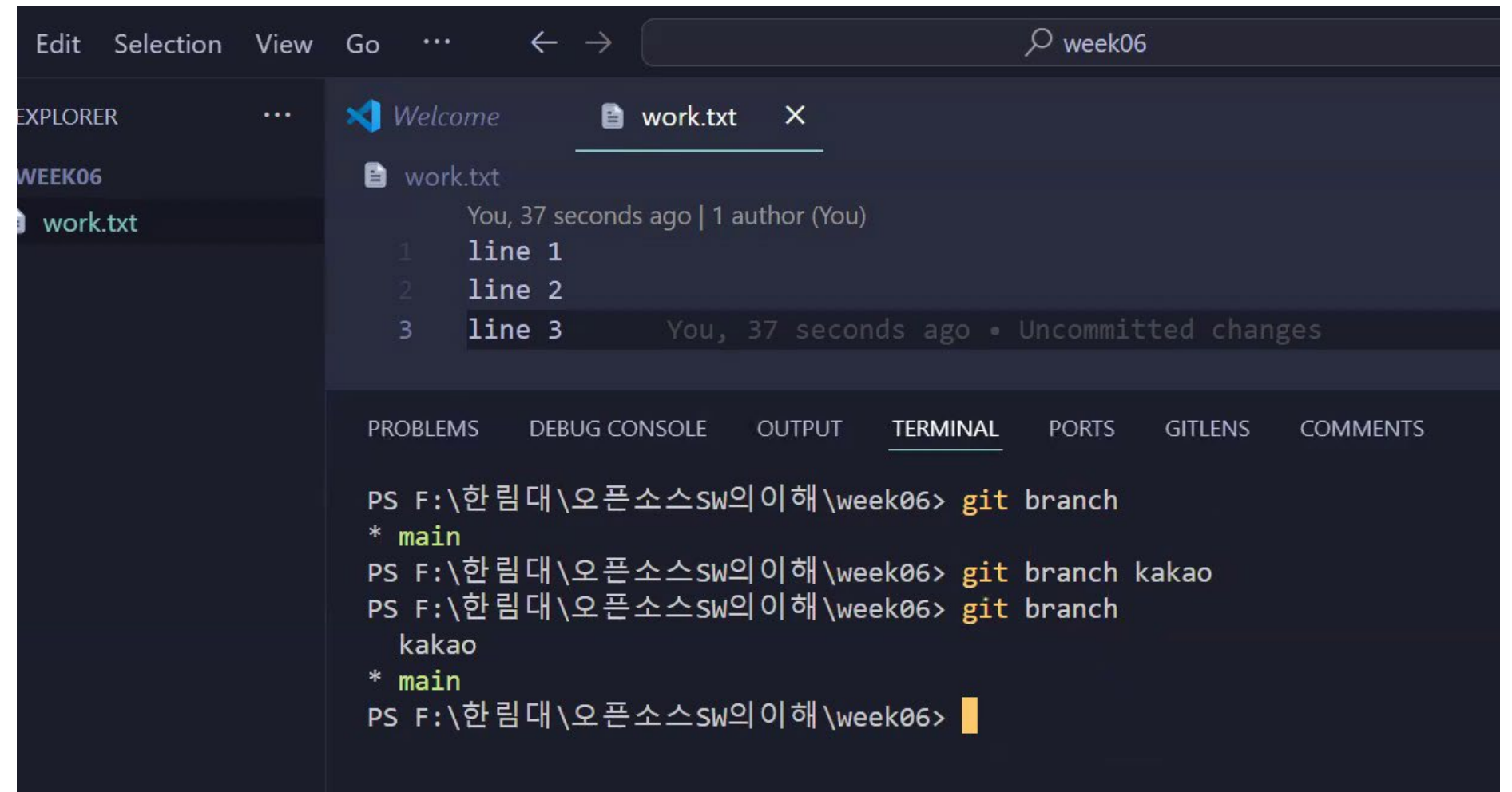
▶ 브랜치 생성하기

❖ 새로운 브랜치 생성

- git branch kakao

❖ 명령어를 실행한 후 `git branch` 입력

❖ 현재 존재하는 브랜치 목록 확인 가능



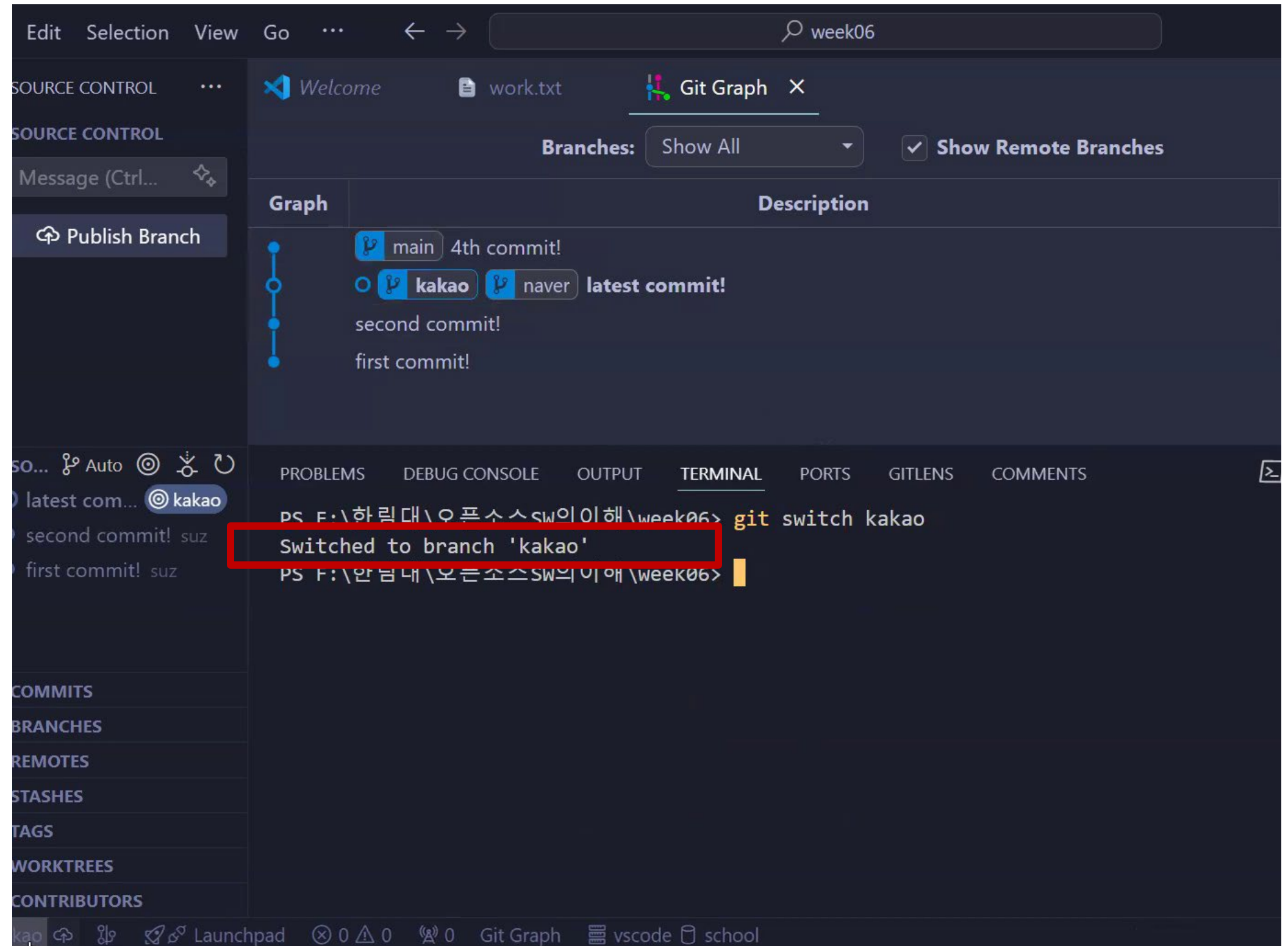
The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows a file named 'work.txt' under a folder 'WEEK06'. The Editor pane shows the content of 'work.txt' with three lines: 'line 1', 'line 2', and 'line 3'. The Terminal pane at the bottom shows the following commands and output:

```
PS F:\한림대\오픈소스SW의이해\week06> git branch
* main
PS F:\한림대\오픈소스SW의이해\week06> git branch kakao
PS F:\한림대\오픈소스SW의이해\week06> git branch
kakao
* main
PS F:\한림대\오픈소스SW의이해\week06> 
```

Branch 변경하기

▶ 브랜치 전환하기

- ① 생성한 브랜치로 전환하기
 - `git switch kakao`



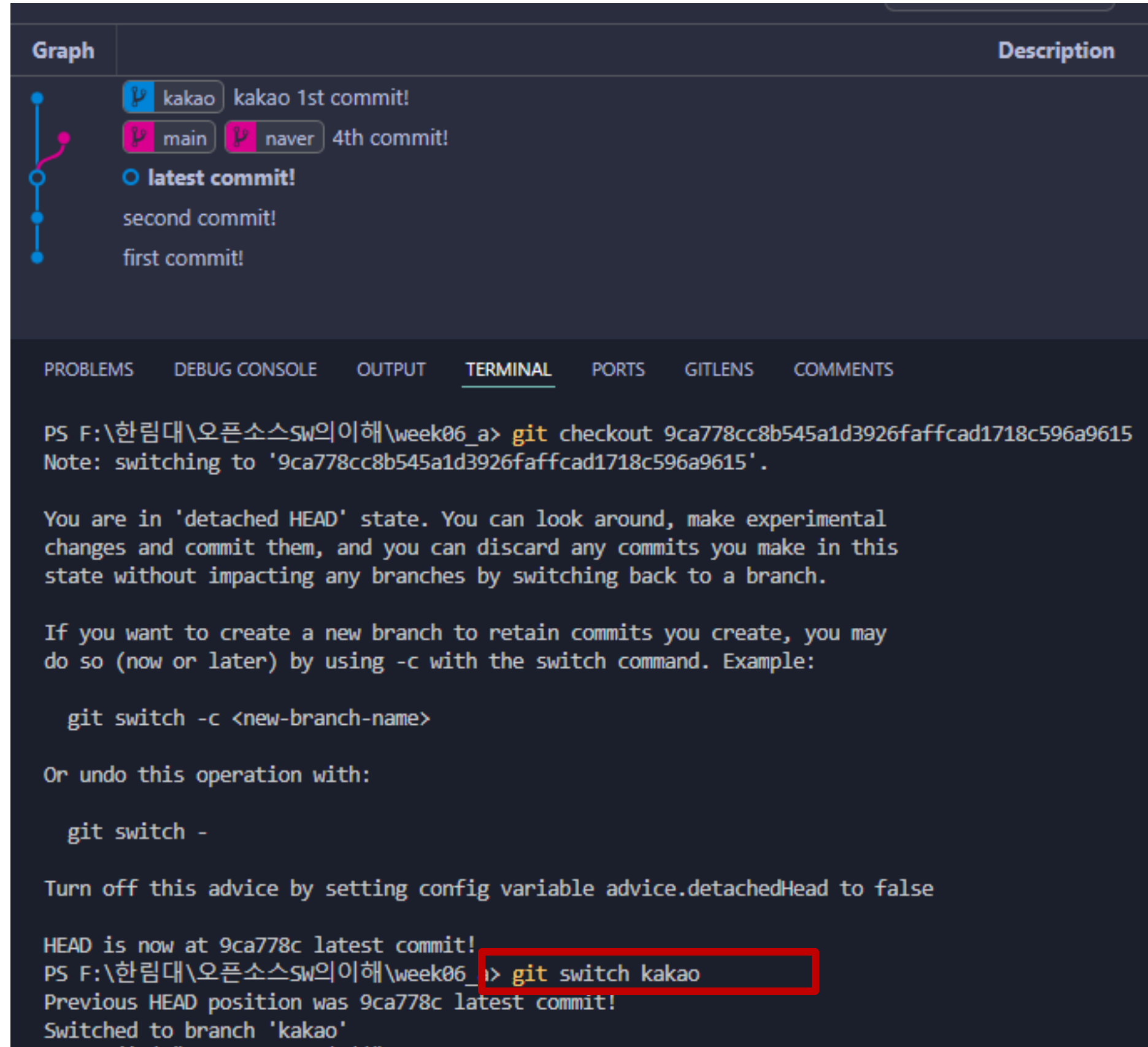
Branch 변경하기

▶ 브랜치 전환하기

※이전 Git 버전에서는 다음과 같이 사용:

- git checkout kakao

또는 다른 커밋으로 이동



```
Graph | Description
-----|-----
kakao | kakao 1st commit!
main  | 4th commit!
naver | 4th commit!
latest commit!
second commit!
first commit!

PROBLEMS | DEBUG CONSOLE | OUTPUT | TERMINAL | PORTS | GITLENS | COMMENTS

PS F:\한림대\오픈소스SW의이해\week06_a> git checkout 9ca778cc8b545a1d3926faffcad1718c596a9615
Note: switching to '9ca778cc8b545a1d3926faffcad1718c596a9615'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 9ca778c latest commit!
PS F:\한림대\오픈소스SW의이해\week06_a> git switch kakao
Previous HEAD position was 9ca778c latest commit!
Switched to branch 'kakao'
```

Branch 변경하기

▶ 브랜치 간의 차이점 비교하기

① 브랜치 사이의 커밋 차이 확인

- `git log main..kakao`

➤ `main` 브랜치에는 없고 `kakao` 브랜치에만 있는 커밋만 표시

반대로, `kakao` 브랜치에는 없고 `main` 브랜치에만 있는 커밋을 보려면?

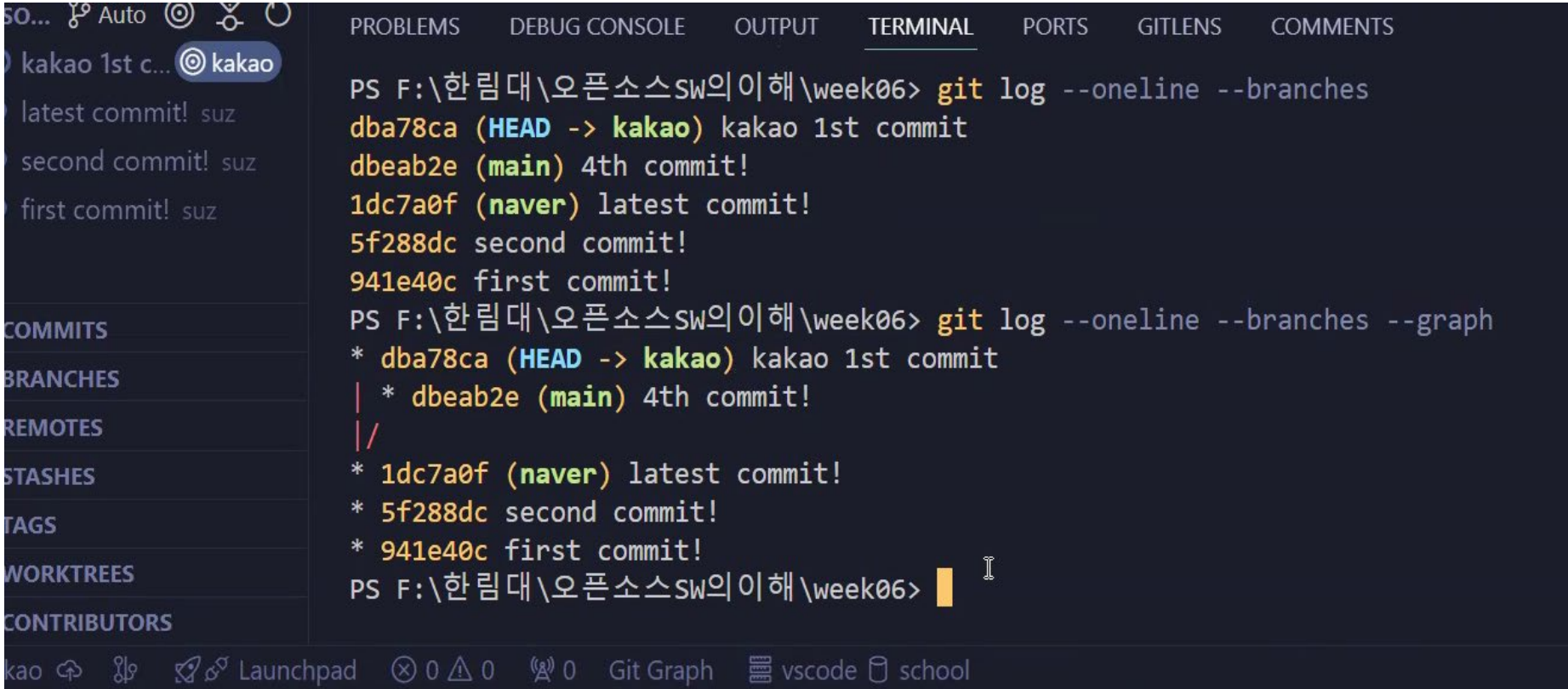
- `git log kakao..main`

Branch 변경하기

▶ 브랜치 간의 차이점 비교하기

로그 한 눈에 파악하기

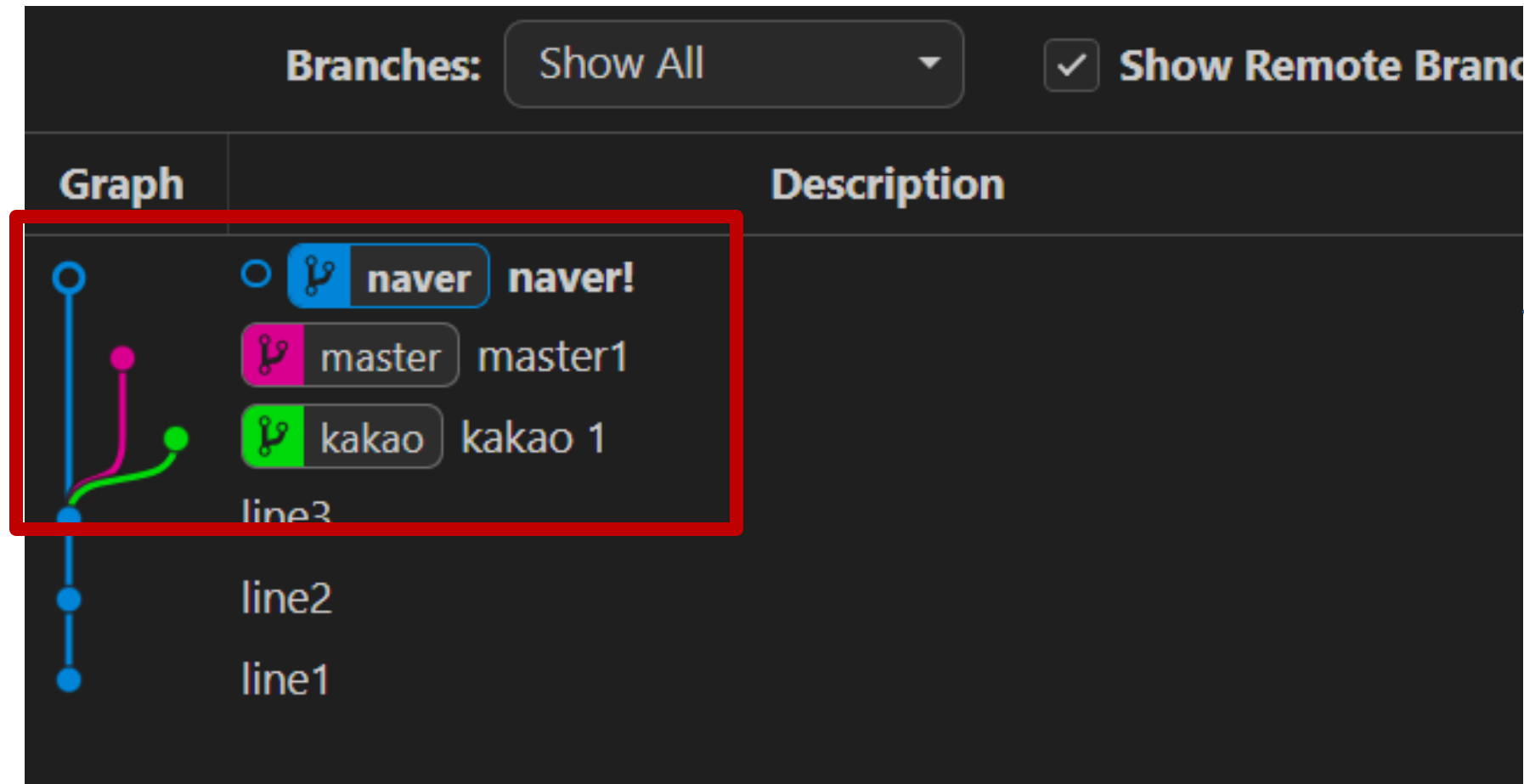
❖ git log --oneline --graph



```
PS F:\한림대\오픈소스SW의이해\week06> git log --oneline --branches
dba78ca (HEAD -> kakao) kakao 1st commit
dbeab2e (main) 4th commit!
1dc7a0f (naver) latest commit!
5f288dc second commit!
941e40c first commit!

PS F:\한림대\오픈소스SW의이해\week06> git log --oneline --branches --graph
* dba78ca (HEAD -> kakao) kakao 1st commit
| * dbeab2e (main) 4th commit!
|/
* 1dc7a0f (naver) latest commit!
* 5f288dc second commit!
* 941e40c first commit!
```

Branch 변경하기



CLASSSUM

① 이런 형태로 구성하기!

- 순서와 색은 상관없음
=> Kakao가 더 위로 와도 됨

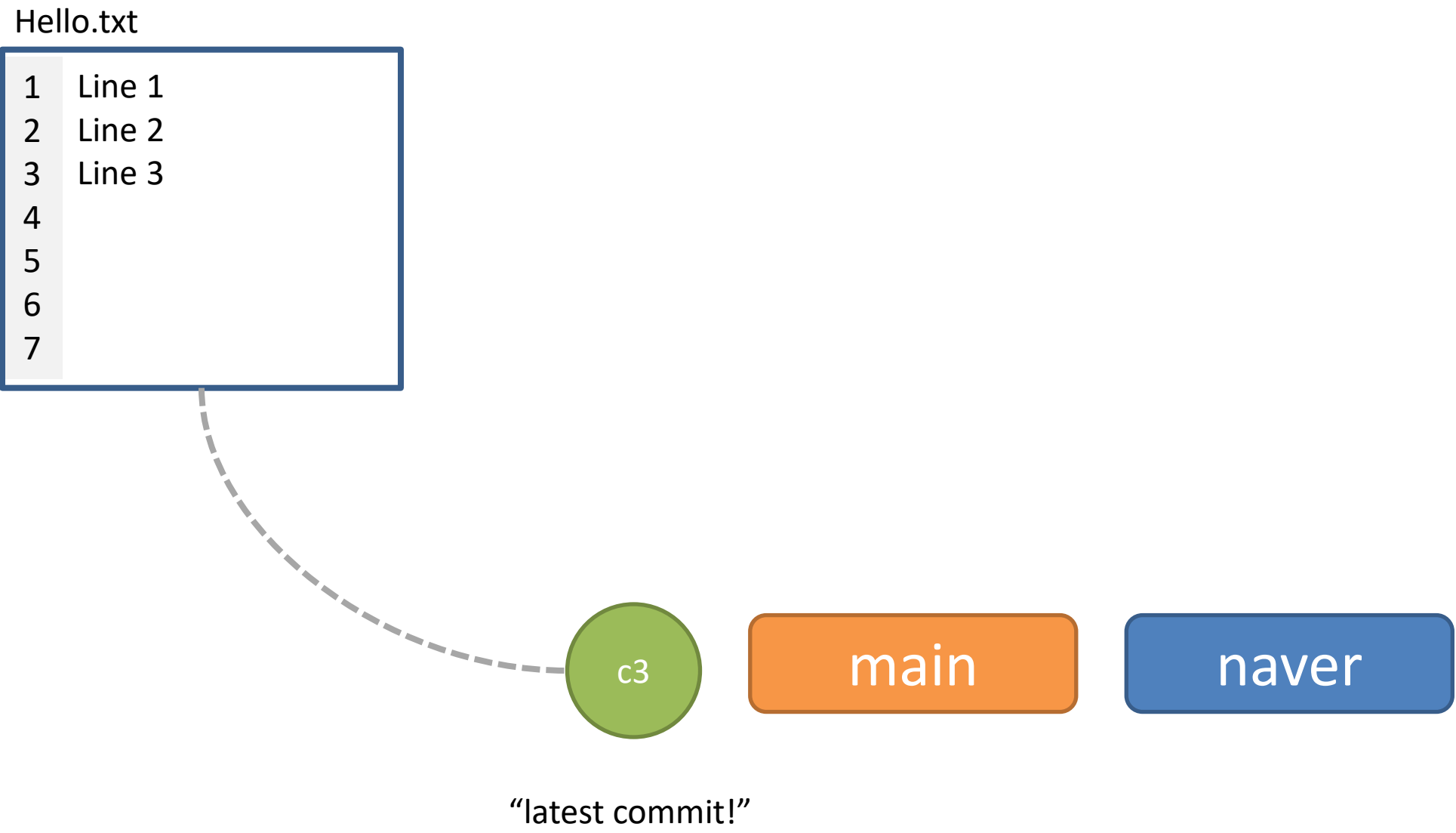
Branch 병합하기

한림대학교

Branch 병합하기



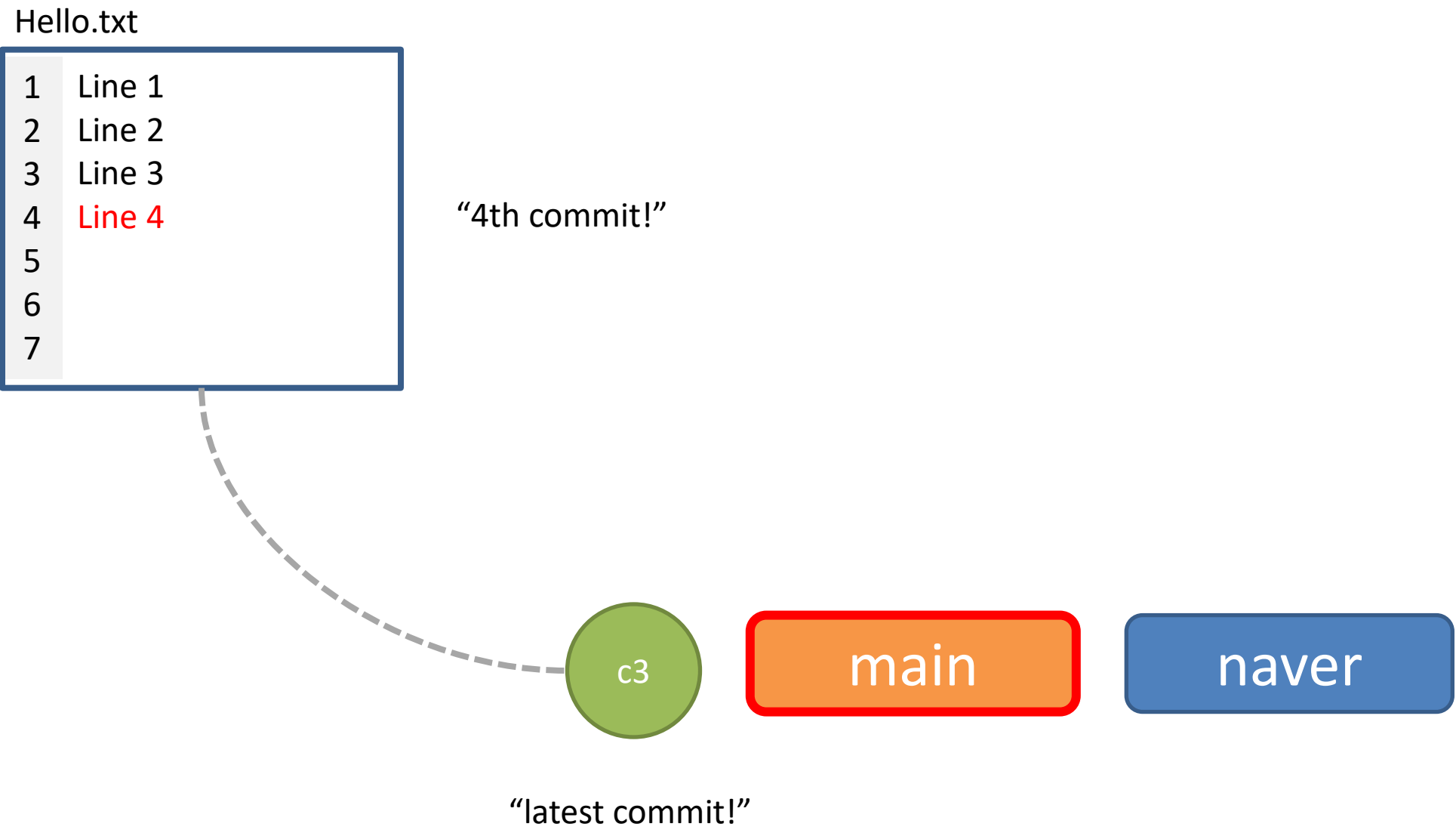
▶ 브랜치 병합하기



Branch 병합하기



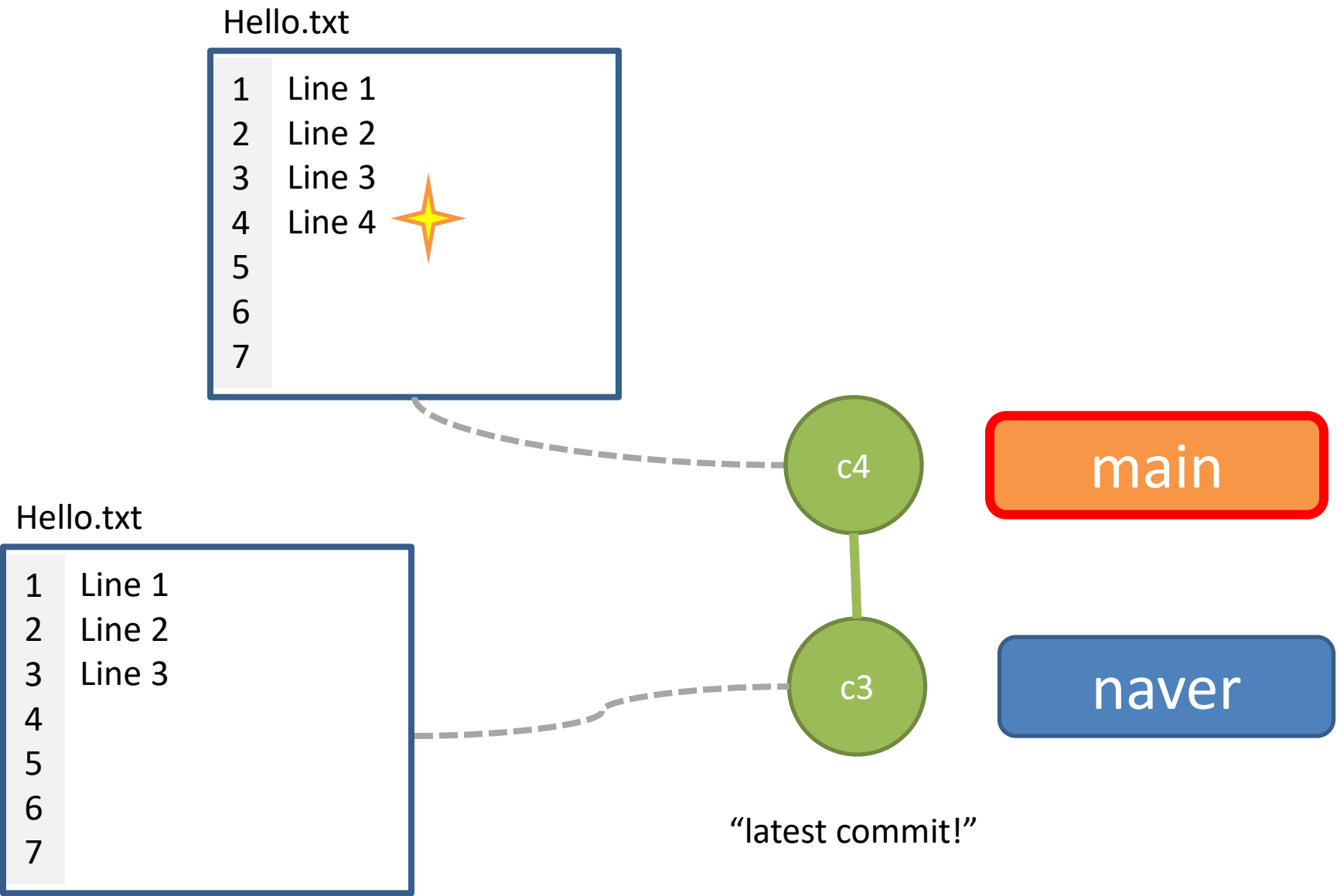
▶ 브랜치 병합하기



Branch 병합하기



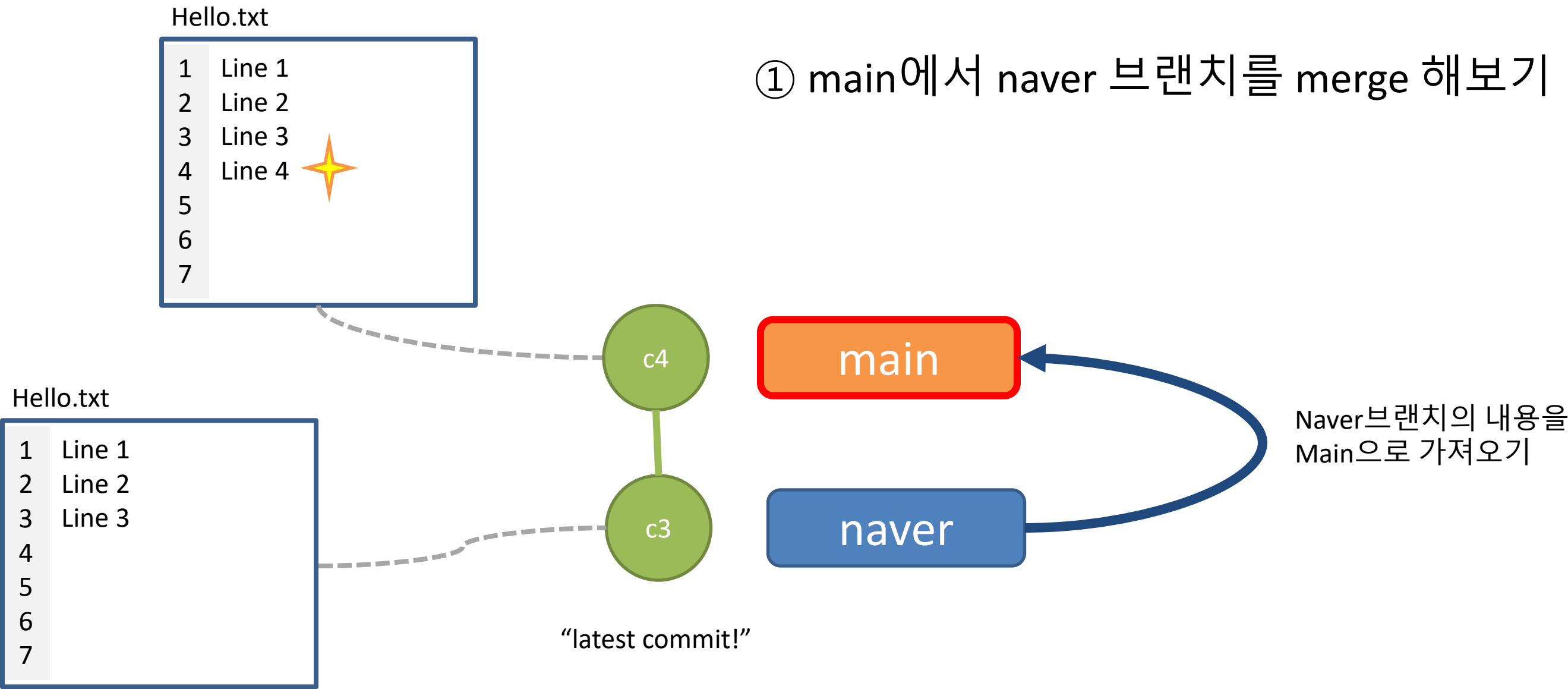
▶ 브랜치 병합하기



Branch 병합하기

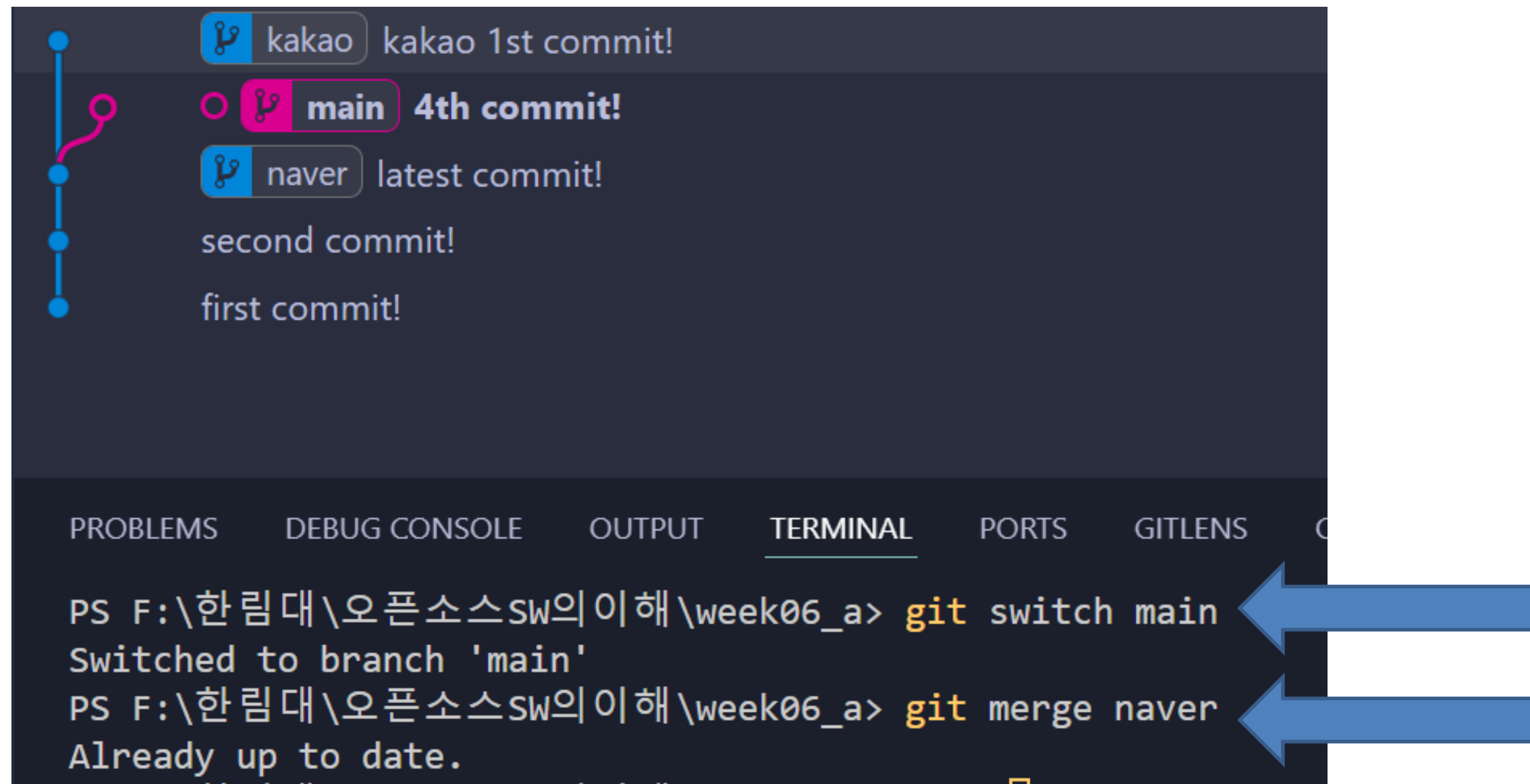


▶ 브랜치 병합하기



Branch 병합하기

▶ Git Branch Merge



The screenshot displays the Git interface in VS Code. The left sidebar shows a commit history with four commits: 'first commit!', 'second commit!', 'naver latest commit!', and 'main 4th commit!'. The 'main' branch is highlighted in pink. The terminal window at the bottom shows the following commands and output:

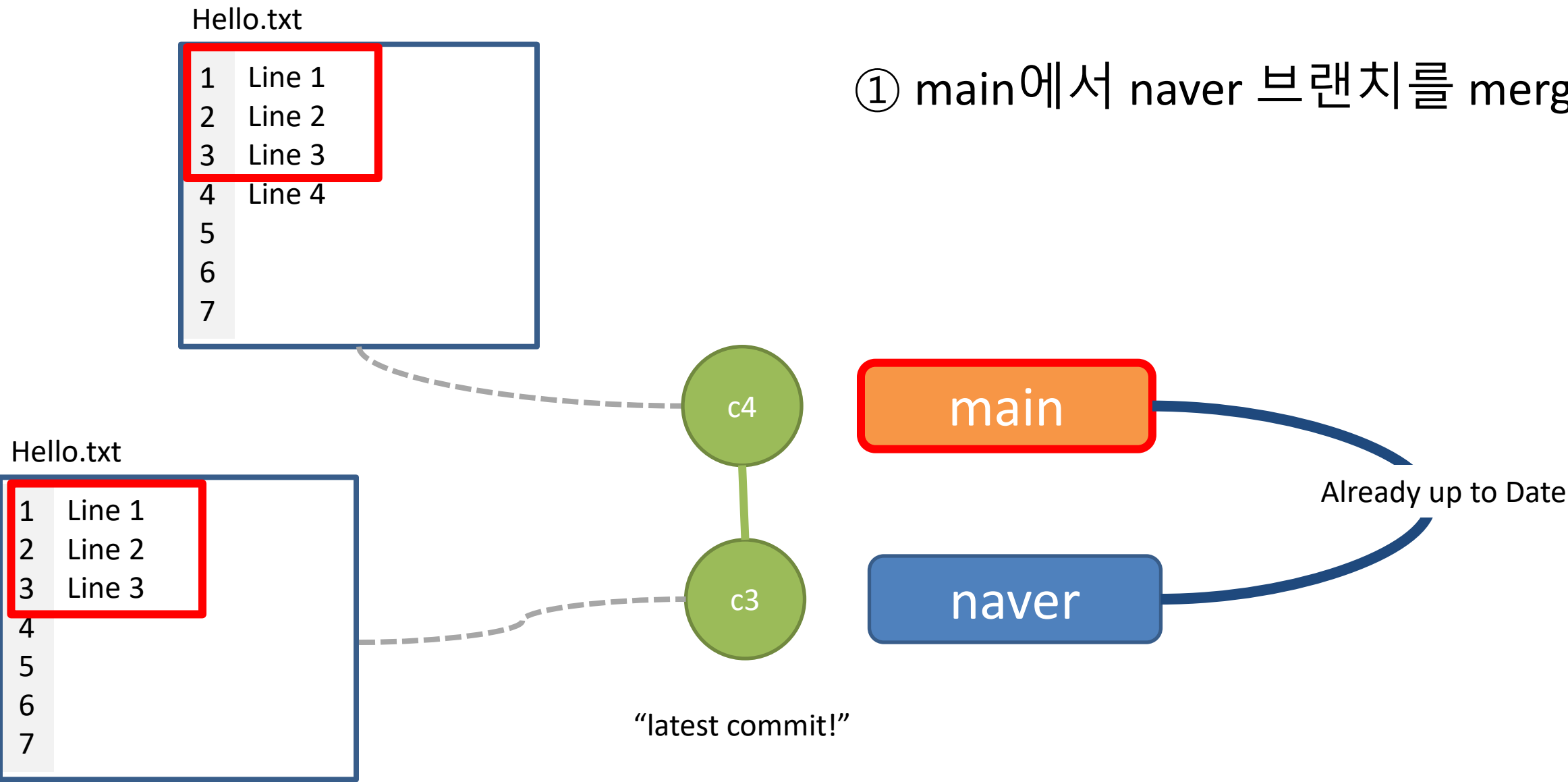
```
PS F:\한림대\오픈소스SW의 이해\week06_a> git switch main
Switched to branch 'main'
PS F:\한림대\오픈소스SW의 이해\week06_a> git merge naver
Already up to date.
```

Two blue arrows point to the terminal output, indicating the successful execution of the `git switch main` and `git merge naver` commands.

Branch 병합하기



▶ 브랜치 병합하기

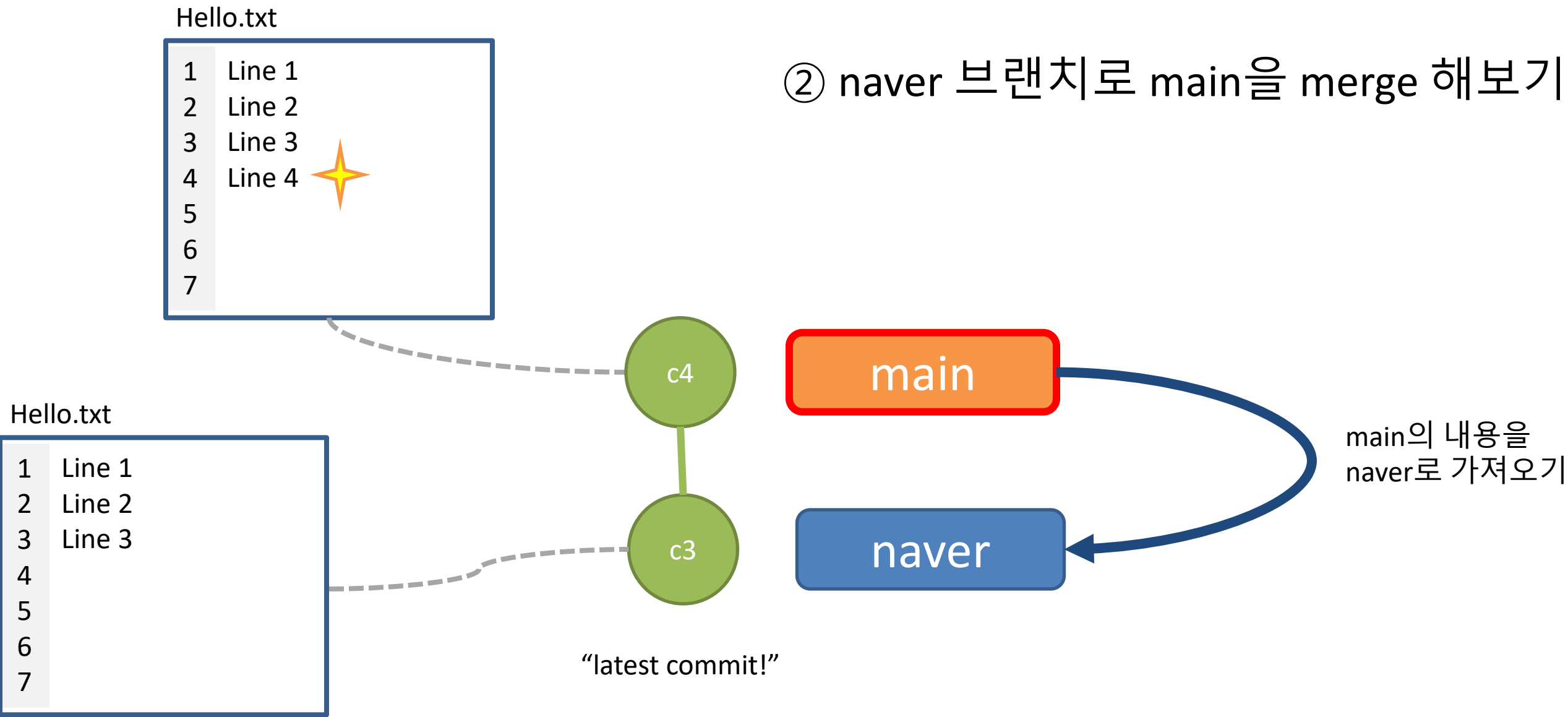


① main에서 naver 브랜치를 merge 해보기

Branch 병합하기



▶ 브랜치 병합하기



Branch 병합하기



▶ Git Branch Merge

Graph

Description

kakao

 kakao 1st commit!

naver

main

 4th commit!

latest commit!

second commit!

first commit!

PROBLEMS

DEBUG CONSOLE

OUTPUT

TERMINAL

PORTS

GITLENS

COMMENTS

```
PS F:\한림대\오픈소스SW의이해\week06_a> git switch naver
Switched to branch 'naver'
PS F:\한림대\오픈소스SW의이해\week06_a> git merge main
Updating 9ca778c..ae62cd2
Fast-forward
 test.txt | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
```

CLASSSUM

- ❖ Fast-Forward (빨리 감기) 병합
 - Branch를 분기한 이후 아무 변화가 없다면 (새로운 커밋x)
Git에서 빠른 병합 가능
 - 포인터만 이동

오늘은 여기까지.

Thank you for listening.